Computer Organization and Assembly Language (EL-2003)

Semester: Fall 2026 | Section: BAI-3A | Instructor: Ms. Sadaf Zehra

LAB # 03 – Practice Tasks (Data Types, Definition Statements & Basic Instructions)

Instructions: Solve the following tasks in the MASM/Irvine32 environment used in your labs. Each task builds directly on the concepts covered in the Lab 03 manual (registers, data definition statements, the DUP operator, and the MOV/ADD/SUB instructions). Write clean, well-commented code and take a screenshot of the output (register dump) where required.

Practice Tasks

Task 1: Meet the Registers

Scenario: You have joined a robotics club, and your senior tells you the robot's control unit uses small "scratchpad" storage spaces called registers instead of slow memory, to keep calculations fast. To understand this before coding the robot's logic, your senior asks you to first explore how the CPU's general-purpose registers are structured.

What to do:

- Write short definitions (1–2 lines each) of EAX, EBX, ECX, and EDX in your own words.
- Show how EAX splits into AX, and then into AH and AL, using a simple diagram or table.
- State one typical use of ECX and one typical use of EDX, based on the manual.

Deliverable: *A short write-up (half a page) with the table/diagram, submitted as part of your lab file.*

Task 2: Setting Up a Student Record

Scenario: You are building a very simple student record system for your department in assembly language. Before writing any instructions, you need to reserve memory for a student's roll number, marks, and name using data definition statements.

What to do:

- Define a variable rollNo of type WORD to store a roll number (e.g., 2201).
- Define a variable marks of type BYTE to store marks out of 100 (e.g., 87).
- Define a variable studentName of type BYTE to store a string containing your name, properly null-terminated.
- Define one uninitialized variable called grade of type BYTE using the ? symbol, to be filled in later.

```
.data
rollNo      WORD   2201
marks       BYTE   87
studentName BYTE   "Ali Raza", 0
grade       BYTE   ?
```

Deliverable: *The .data segment with all four variables correctly declared, with a comment next to each line explaining what it stores.*

Task 3: Building a Weekly Attendance List

Scenario: Your class representative wants a simple way to store how many students attended class each day of a 5-day week (Monday to Friday), so it can later be processed by a program.

What to do:

- Declare a BYTE array named attendance that stores 5 values, one for each day, using multiple initializers in a single statement (e.g., 30, 28, 32, 25, 29).

- Declare a second BYTE array named emptySeats that reserves 5 uninitialized bytes (values not known yet).
- Add a comment above each line explaining what the array represents.

Deliverable: *Both array declarations with comments, ready to paste into a .asm file.*

Task 4: Reserving Space with DUP

Scenario: You are asked to reserve memory space for a message board feature and a buffer area in a small console application, but writing out every byte manually would be tedious. Your instructor suggests using the DUP operator instead.

What to do:

- Using DUP, reserve 15 bytes, all initialized to 0, named buffer.
- Using DUP, reserve 10 bytes that remain uninitialized, named tempSpace.
- Using DUP, create a variable named pattern that repeats the string "HI" exactly 5 times (so it reads "HIHIHIHIHI").
- In one sentence, explain why DUP is more convenient than typing every value separately.

Deliverable: *Three DUP-based declarations plus your one-sentence explanation.*

Task 5: ASCII Code Explorer

Scenario: While debugging a program, your lab partner is confused about why MOV BX, 'A' and MOV BX, 41h give the exact same result. You decide to prove this to them with a short demonstration.

What to do:

- Write four different MOV instructions that all load the ASCII value of the letter 'B' into register CX — one using the character constant, one using hexadecimal, one using decimal, and one using binary.
- Add a comment next to each instruction stating the value being loaded.
- In 1–2 lines, explain in your own words why all four instructions produce the same result in CX.

Deliverable: *The four MOV instructions with comments, plus your short explanation.*

Task 6: Tuck Shop Cash Register

Scenario: You are helping the university's small tuck shop build a very basic cash calculator in assembly. At the start of the day, the shop has an opening cash balance. During the day, some cash comes in from sales, and some cash goes out for buying supplies. You need to calculate the final balance at the end of the day.

What to do:

- Using the sample program structure from the manual (AddSub.asm) as a guide, write a complete program that:
- Loads an opening balance of 5000 into EAX using MOV.
- Adds sales income of 1500 to EAX using ADD.
- Subtracts supply expenses of 800 from EAX using SUB.
- Calls DumpRegs to display the final register values, then exits.
- Before running it, manually calculate what the final value in EAX should be, then verify your program's output matches.

```
TITLE Tuck Shop Cash Register (CashRegister.asm)
INCLUDE Irvine32.inc
.code
main PROC
    ; TODO: load opening balance
    ; TODO: add sales income
    ; TODO: subtract supply expenses
    ; TODO: display registers and exit
```

```
main ENDP
END main
```

Deliverable: *The completed .asm program, your hand-calculated expected final balance, and a screenshot of the DumpRegs output showing EAX matches your calculation.*